



Manual Técnico

JOSNISHOP

I. Introducción

El propósito de este documento es detallar la arquitectura, el diseño, y los procedimientos de mantenimiento del sistema de comercio electrónico **JOSNISHOP**.

Audiencia: Este manual está dirigido a desarrolladores, ingenieros de operaciones (DevOps) y personal de soporte técnico.

1.1. Stack Tecnológico

El proyecto JOSNISHOP utiliza una arquitectura Cliente-Servidor con las siguientes tecnologías:

- Frontend (Interfaz de Usuario):
 - Tecnología: React + TypeScript
 - Función: Maneja la interfaz gráfica, la experiencia de vídeos, búsqueda y carrito.
- Backend (API de Servicios):
 - Tecnología: FastAPI + Python
 - Función: Ejecuta la lógica de negocio, gestiona el inventario, la autenticación y la base de datos.
- Base de Datos (Persistencia):
 - Tecnología: [PENDIENTE: Insertar aquí la tecnología de BD (ej. PostgreSQL, MongoDB)]
- Control de Versiones:
 - Tecnología: Git

II. Arquitectura y Diseño del Sistema

2.1. Modelo de Datos (Esquema Básico)

El sistema se organiza alrededor de entidades clave que gestionan la operación del e-commerce:

- Usuario: Gestiona la autenticación y los perfiles de cliente.



- Producto: Almacena información detallada del artículo (nombre, precio, stock, descripción).
- Pedido (Orden): Registra las transacciones de compra.
- Carrito: Relaciona temporalmente al usuario con los productos seleccionados antes de la compra.

2.2. Diccionario de Datos: Definición de Tablas

1. Tabla: Roles

- Descripción: Define los niveles de acceso y permisos dentro del sistema.
- Campos: id_rol (PK), nombre, estado.

2. Tabla: Usuarios

- Descripción: Almacena la información de perfil y credenciales para el acceso de clientes.
- Campos: id_usuario (PK), nombre, correo, contraseña, rol_id (FK), estado.

3. Tabla: Categorías

- Descripción: Agrupación lógica de los productos para facilitar la navegación en la interfaz.
- Campos: id (PK), nombre, estado, fecha_creacion.

4. Tabla: Productos

- Descripción: Almacena la información comercial básica de cada artículo, como su nombre y descripción detallada.
- Campos: id (PK), nombre, descripcion, categoria_id (FK).

5. Tabla: Inventarios

- Descripción: Control de existencias físicas, permitiendo gestionar el stock mínimo para alertas de reposición.
- Campos: id (PK), producto_id (FK), cantidad, stock_minimo, fecha_actualizacion.

6. Tabla: Item

- Descripción: Registro individualizado por serial de cada unidad de producto, incluyendo su historial de ingreso, salida y precio de venta actual.



- Campos: id (PK), serial, fecha_ingreso, fecha_salida, estado, precio, productos_id (FK).

7. Tabla: Pedidos

- Descripción: Encabezado de la transacción que vincula al cliente con la fecha y el monto total de la compra.
- Campos: id_pedido (PK), cliente_id (FK), fecha_pedido, estado, total.

8. Tabla: Detalle_Pedido

- Descripción: Desglose de los productos incluidos en cada pedido, calculando cantidades y subtotales por línea de artículo.
- Campos: id_detalle (PK), pedido_id (FK), producto_id (FK), cantidad, subtotal.

9. Tabla: Bot_Responses

- Descripción: Base de conocimientos para el soporte automatizado, gestionando respuestas por palabras clave y prioridad.
- Campos: clave, respuesta, prioridad, enabled.

10. Tabla: Videos

- Descripción: Almacena las rutas de los videos promocionales utilizados en el carrusel principal de la aplicación.
- Campos: producto_id (FK), url, fecha_subida.

11. Tabla: Reseñas

- Descripción: Feedback de los clientes sobre los productos, incluyendo calificación numérica y comentarios.
- Campos: id_reseña (PK), producto_id (FK), cliente_id (FK), calificación, comentario, respuesta_vendedor, fecha.

12. Tabla: Pagos

- Descripción: Registro de la información transaccional para el procesamiento de cobros de forma segura.
- Campos: id_pago (PK), id_usuario (FK), nombre_tarjeta, numero_tarjeta, fecha_expiracion, cvv, fecha_pago.



2.3. Diseño de la API (Backend - FastAPI)

El Backend expone una API RESTful, organizada en controladores que agrupan las funcionalidades.

- Controladores Principales:
 - usuario_controller.py: Maneja el registro, inicio de sesión y perfiles.
 - producto_controller.py: Rutas para obtener, crear y actualizar productos.
 - orden_controller.py: Gestión del proceso de pedido y checkout.
- Convenciones: Todas las rutas siguen la convención REST (ej. GET /api/v1/productos, POST /api/v1/pedidos).

III. Estándares de Codificación y Calidad

El proyecto se rige por la guía Guia_EstandaresCodigo.md, aplicando reglas específicas para cada entorno.

3.1. Reglas de Nomenclatura y Estilo

Componente	Regla	Ejemplo
Frontend Componentes (TS/React)	PascalCase	InicioSesion.tsx
Frontend Variables/Funciones	camelCase	userName, handleSubmit
Backend Clases (Python)	PascalCase	UsuarioController
Backend Variables/Funciones (Python)	snake_case	get_productos, usuario_origen

3.2. Herramientas de Calidad (Linting y Formateo)

El cumplimiento de los estándares es automático mediante estas herramientas:



- Frontend (TS/React):
 - ESLint: Análisis estático para detectar errores y problemas de estilo.
 - Prettier: Formateador de código con indentación de 2 espacios.
- Backend (Python/FastAPI):
 - Black: Formateador de código que adhiere al estándar PEP 8 (indentación de 4 espacios).
 - isort: Herramienta para ordenar y clasificar automáticamente las importaciones.
 - ruff: Linter de alto rendimiento para aplicar reglas de estilo Python.

3.3. Documentación Interna

- Frontend: Uso de JSDoc para documentar componentes, propiedades y funciones.
- Backend: Uso de Docstrings triples ("""...""") en clases y funciones para describir su propósito y parámetros.

IV. Entorno de Desarrollo y Configuración

Esta sección detalla los pasos para poner en marcha una instancia local del proyecto.

4.1. Prerrequisitos

Para ejecutar el proyecto, se requiere el siguiente software instalado:

- Node.js (versión requerida: [PENDIENTE: Especificar versión]).
- Python 3 (versión requerida: [PENDIENTE: Especificar versión]).
- Gestor de Base de Datos local (si aplica, ej. Docker, instalación de PostgreSQL).
- Git.

4.2. Pasos de Instalación Local

A. Configuración del Backend (Python/FastAPI)

1. Clonar el repositorio: `git clone https://github.com/nicolamaya/JOSNISHOP.git`
2. Navegar a la carpeta del backend: `cd josnishop/BACKEND`
3. Crear y activar el entorno virtual: `python -m venv venv` y `source venv/bin/activate`
4. Instalar dependencias: `pip install -r requirements.txt`
5. Ejecutar el servidor: `uvicorn main:app --reload` (o comando de inicio).



B. Configuración del Frontend (React/TypeScript)

1. Navegar a la carpeta del frontend: `cd joshishop/frontend`
2. Instalar dependencias: `npm install` (o `yarn install`).
3. Iniciar la aplicación: `npm run dev` (o comando de inicio).

4.3. Variables de Entorno

El sistema Joshishop utiliza archivos `.env` para gestionar configuraciones sensibles, conexiones externas y límites de seguridad. Esta información permite que la aplicación funcione correctamente sin exponer credenciales directamente en el código fuente.

A. Configuración de Correo y Notificaciones (SMTP)

Estas variables permiten que el sistema envíe correos electrónicos automáticos (como confirmaciones de registro o recuperación de contraseñas).

- `SMTP_EMAIL`: Cuenta de correo emisora (`joshishop@gmail.com`).
- `SMTP_PASSWORD`: Contraseña de aplicación para acceso al servidor de correo.
- `SMTP_SERVER`: Dirección del servidor de salida (`smtp.gmail.com`).
- `SMTP_PORT`: Puerto de conexión segura (465).

B. Persistencia de Datos (MySQL)

- `DATABASE_URL`: Define la conexión a la base de datos MySQL. Incluye el driver (`pymysql`), el usuario (`root`), la contraseña (`admin`), el host local y el puerto (3315).

C. Seguridad y Autenticación (JWT)

Parámetros críticos para la gestión de sesiones de usuario y cifrado de datos.

- `SECRET_KEY`: Clave secreta para firmar los tokens de acceso.
- `ALGORITHM`: Algoritmo de cifrado utilizado (HS256).
- `ACCESS_TOKEN_EXPIRE_MINUTES`: Tiempo de expiración del token de acceso (30 minutos).
- `REFRESH_TOKEN_EXPIRE_DAYS`: Tiempo de validez del token de refresco (7 días).

D. Interacción entre Servicios (CORS)



Define qué direcciones externas tienen permiso para realizar peticiones a la API del Backend.

- **FRONTEND_URL**: Dirección de la interfaz de usuario en desarrollo (<http://localhost:5173>).
- **ALLOWED_ORIGINS**: Lista de dominios autorizados para interactuar con la API.

E. Seguridad y Límites de la Aplicación

Controles para prevenir abusos y ataques de denegación de servicio (DoS) o fuerza bruta.

- **ENVIRONMENT**: Define el modo de ejecución del sistema (development).
- **MAX_LOGIN_ATTEMPTS**: Límite de 5 intentos de inicio de sesión antes de aplicar un bloqueo temporal.
- **MAX_FILE_SIZE_MB**: Límite de carga de archivos de hasta 5MB.
- **MAX_VIDEO_DURATION_SECONDS**: Restricción de 60 segundos para videos promocionales.

F. Control de Tráfico (Rate Limiting)

- **RATE_LIMIT_...**: Serie de variables que limitan la cantidad de peticiones permitidas por ventana de tiempo en endpoints sensibles (Login, Registro y API general) para garantizar la estabilidad del servidor.

V. Mantenimiento y Operaciones

5.1. Procedimientos de Despliegue (Deployment)

El despliegue de Josnishop requiere la preparación independiente tanto del servidor de API (Backend) como de la interfaz de usuario (Frontend), asegurando que ambos puedan comunicarse a través de los protocolos definidos.

A. Preparación del Servidor Backend (FastAPI)

Para poner en producción el servidor de Python, se deben seguir estos pasos:

1. **Actualización del Código**: Descargar la última versión estable desde el repositorio principal mediante el comando `git pull origin main`.



2. Entorno Virtual: Asegurarse de que el entorno virtual esté activo y las dependencias actualizadas con `pip install -r requirements.txt`.
3. Migraciones de Base de Datos: Ejecutar los scripts de actualización de tablas en MySQL para asegurar que la estructura coincida con el código actual.

B. Construcción y Despliegue del Frontend (React)

El frontend debe ser "compilado" para generar archivos estáticos que cualquier navegador pueda leer rápidamente.

1. Instalación de Dependencias: Ejecutar `npm install` para asegurar que todas las librerías estén presentes.
2. Generación del Build: Ejecutar el comando `npm run build`. Esto creará una carpeta llamada `/dist` o `/build` con los archivos optimizados.
3. Servidor de Estáticos: Los archivos resultantes deben ser cargados en un servidor web (como Nginx o Apache) o plataformas de hosting (como Vercel o Netlify).
4. Verificación de URL de API: Es crítico que la variable `REACT_APP_API_URL` apunte a la dirección IP o dominio real del servidor Backend y no a `localhost`.

C. Integración Continua (CI/CD) - Opcional

Si el proyecto utiliza herramientas como GitHub Actions, el proceso se automatiza de la siguiente manera:

- Cada vez que se realiza un "Push" a la rama principal (main), se disparan pruebas automáticas de calidad (Linters).
- Si las pruebas pasan, el sistema genera automáticamente el build del frontend y reinicia el servicio del backend en el servidor de producción.

D. Verificación Post-Despliegue

Una vez finalizado el proceso, se deben realizar las siguientes pruebas de humo:

1. Verificar que el login funcione correctamente (comunicación con la BD).
2. Cargar la interfaz de videos para asegurar que los archivos multimedia se visualicen.
3. Realizar una búsqueda de producto para validar los controladores de FastAPI.

5.2. Monitoreo y Logging



Para garantizar la estabilidad de Josphop, el sistema cuenta con mecanismos de registro, que permiten rastrear eventos, errores de usuario y fallos del servidor en tiempo real.

A. Registro de Errores en el Backend (Logging)

El backend utiliza el sistema de logging nativo de Python y las capacidades de FastAPI/Uvicorn para documentar la actividad.

- Ubicación de los Logs: Por defecto, los registros se emiten a través de la salida estándar (stdout) y la salida de errores (stderr). En entornos de producción Linux, estos suelen ser capturados por el servicio de gestión (ej. `journalctl -u josphop.service`).
- Niveles de Log:
 - INFO: Registro de peticiones exitosas (ej. inicios de sesión, carga de productos).
 - WARNING: Advertencias de seguridad (ej. intentos de login fallidos que superan el `MAX_LOGIN_ATTEMPTS`).
 - ERROR/CRITICAL: Fallos en la conexión a la base de datos MySQL o errores en el envío de correos SMTP.

B. Monitoreo de Seguridad y Tráfico

Gracias a las variables de entorno configuradas (Rate Limiting), el sistema monitorea:

- Abuso de Endpoints: Se registran las IPs que exceden el `RATE_LIMIT_API_MAX_REQUESTS` para prevenir ataques de denegación de servicio.
- Estado de la Base de Datos: Monitoreo constante de la latencia en las consultas a las tablas productos e inventarios.

C. Monitoreo en el Frontend

En el cliente (React), los errores se capturan mediante:

- Consola de Desarrollador: Registro de errores de red (CORS) o fallos en la renderización de los videos del carrusel.
- Validación de API: Si la variable `REACT_APP_API_URL` no responde, el sistema está programado para mostrar un mensaje de "Servicio no disponible" al usuario.



D. Herramientas Sugeridas para Escalamiento

Para una etapa avanzada del proyecto, se recomienda la integración de:

1. Sentry: Para la captura automática de excepciones en el código Python y errores de JavaScript en el navegador del cliente.
2. Grafana/Prometheus: Para visualizar gráficas de tráfico y consumo de recursos (CPU/RAM) del servidor.